

.NET framework and C# basics

Contents

.NET framework basics	2
Common Language Runtime	3
Steps in CLR Code Execution Process	3
Detailed Workflow	6
The Default Thread	7
Types of Jit	8
1. Pre-JIT (Pre-JIT Compilation)	8
2. Econo-JIT (Economy JIT)	8
3. Normal-JIT (Standard JIT or JIT Compilation)	8
Example of JIT Compilation in .NET	9
Summary	9
(AOT) compilation	10
AOT Compilation Methods in .NET	10
AOT vs. JIT Compilation	11
Summary	11
C# Language basics	14
String formatting	18
String verbatim	19
String Interpolation	20
typeof Operator	20
Ternary operator	20
Nullable types in C#	21
null-coalescing operator	21
References	22
Assignment	23
Self-Check	27

.NET framework basics

What is .NET framework? Languages in .NET framework?

.NET Framework is a structured, reusable set of tools, libraries, and guidelines that helps developers build software more easily and consistently and is a software framework developed by Microsoft. It provides a controlled environment for developing and running applications.

What happens when application is compiled and executed?

When we Compile one .NET Program with the Appropriate Compiler, the compilation will result into an assembly.

The Assembly can be either in the .dll (or) .exe format.

These Assemblies will Contain the Code in M/s Intermediate Language Format.

so One Assembly is a Collection of MSIL Classes and MetaData.

.Dll stands for Dynamic Link Library. It will be used as Library Application in .NET, Whereas .Exe is an executable so they must contain starting point of execution(main)

the default target of .net compilers is executable

to Execute these Assemblies, the System Must have Common Language Runtime.

It Provides Managed Runtime Infrastructure for the .NET Applications.

Some of CLR Roles are

- 1)It Convert the Requested Assemblies MSIL Code into Native code through its JIT Compiler
- 2)It Optimises the performance of .NET Applications through Memory Management
- 3)It performs various Code Checks and so on.....

C#(.cs) Vb(.vb)

|Compile

|

Assembly(.exe/.dll)----(MSIL/CIL)

|

|CLR(JIT compiler)

|

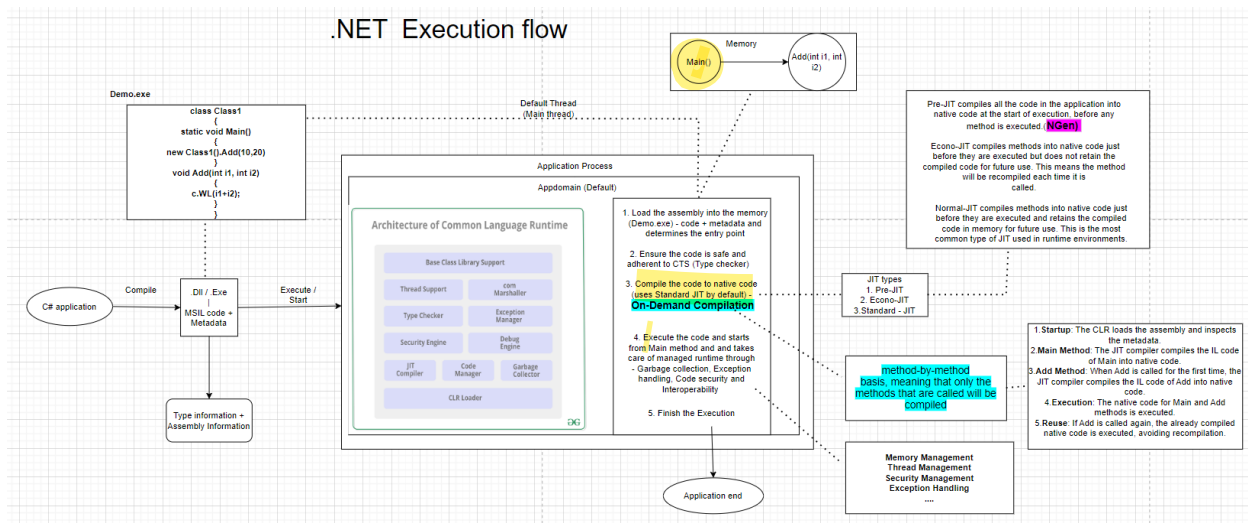
native code

|

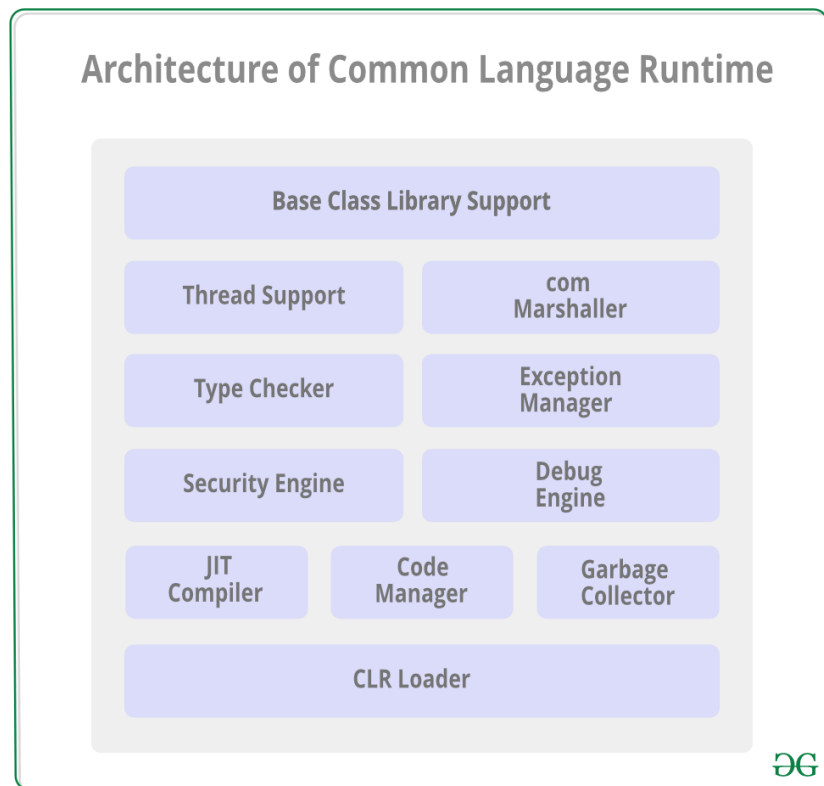
O/S

Common Language Runtime

The Common Language Runtime (CLR) in .NET provides a managed execution environment for .NET applications. The CLR handles various aspects of execution, including memory management, security, and exception handling. Here's a detailed breakdown of the CLR code execution process:



Architecture of Common Language Runtime



There are multiple components in the architecture of Common Language Runtime.

- **Base Class Library Support:** The Common Language Runtime provides support for the base class library. The BCL contains multiple libraries that provide various features such as *Collections, I/O, XML, DataType* definitions, etc. for the multiple .NET programming languages.
- **Thread Support:** The CLR provides thread support for managing the parallel execution of multiple threads. The *System.Threading* class is used as the base class for this.
- **COM Marshaller:** Communication with the COM (Component Object Model) component in the .NET application is provided using the COM marshaller. This provides the COM interoperability support.
- **Type Checker:** Type safety is provided by the type checker by using the Common Type System (CTS) and the Common Language Specification (CLS) that are provided in the CLR to verify the types that are used in an application.
- **Exception Manager:** The exception manager in the CLR handles the exceptions regardless of the .NET Language that created them. For a particular application, the catch block of the exceptions are executed in case they occur and if there is no catch block then the application is terminated.
- **Security Engine:** The security engine in the CLR handles the security permissions at various levels such as the code level, folder level, and machine level. This is done using the various tools that are provided in the .NET framework.
- **Debug Engine:** An application can be debugged during the run-time using the debug engine. There are various ICorDebug interfaces that are used to track the managed code of the application that is being debugged.
- **JIT Compiler:** The [JIT compiler](#) in the CLR converts the Microsoft Intermediate Language (MSIL) into the machine code that is specific to the computer environment that the JIT compiler runs on.

The compiled MSIL is stored so that it is available for subsequent calls if required.

- **Code Manager:** The code manager in CLR manages the code developed in the .NET framework i.e. the managed code. The managed code is converted to intermediate language by a language-specific compiler and then the intermediate language is converted into the machine code by the Just-In-Time (JIT) compiler.
- **Garbage Collector:** Automatic memory management is made possible using the garbage collector in CLR. The garbage collector automatically releases the memory space after it is no longer required so that it can be reallocated.
- **CLR Loader:** Various modules, resources, assemblies, etc. are loaded by the CLR loader. Also, this loader loads the modules on demand if they are actually required so that the program initialization time is faster and the resources consumed are lesser.

Steps in CLR Code Execution Process

1. Source Code Compilation:

- When you compile your C# (or other .NET language) source code using a compiler (such as csc.exe for C#), it gets translated into Microsoft Intermediate Language (MSIL or IL) and stored in an assembly (a .dll or .exe file).

2. Assembly Loading:

- When you run the application, the CLR loads the necessary assemblies into memory. This involves reading the metadata and IL code contained in the assembly files.

3. Verification:

- The CLR verifies the IL code to ensure it is safe to execute. This process involves checking the code for type safety and ensuring that it adheres to the Common Type System (CTS) rules.

4. JIT Compilation:

- **On-Demand Compilation:** The JIT compiler translates IL code into native machine code just before execution. This process occurs on a method-by-method basis, meaning that only the methods that are called will be compiled.
- The JIT compiler also applies various optimizations during this translation to improve performance.

5. Execution:

- The native code is executed by the CPU. During execution, the CLR provides services such as memory management (through garbage collection), exception handling, and security enforcement.

6. Garbage Collection:

- The CLR automatically manages memory allocation and deallocation. The garbage collector (GC) periodically frees up memory occupied by objects that are no longer in use, preventing memory leaks and optimizing the use of available memory.

7. Exception Handling:

- The CLR provides a structured way to handle exceptions using try, catch, and finally blocks. It ensures that exceptions are propagated properly and that resources are cleaned up.

8. Security:

- The CLR enforces security through code access security (CAS) and other mechanisms. It ensures that the code has the necessary permissions to perform certain operations and that it adheres to security policies.

9. Interoperability:

- The CLR provides support for interoperability with unmanaged code, such as calling native APIs or integrating with COM components. This allows .NET applications to interact with existing libraries and systems.

Detailed Workflow

1. Application Start:

- The operating system loader loads the CLR into the process space of the application.

2. Metadata Inspection:

- The CLR inspects the assembly's metadata to determine the entry point (such as the Main method in a C# application).

3. Class Loading:

- The CLR loads the necessary classes and initializes static fields and constructors as required.

4. JIT Compilation and Execution:

- When the entry point method is called, the JIT compiler compiles the IL code of the method into native code.
- The native code is executed, and the CLR continues to monitor the execution flow, JIT-compiling additional methods as they are called.

5. Runtime Services:

- Throughout the execution, the CLR provides various services:
 - **Memory Management:** Allocates and deallocates memory, performs garbage collection.
 - **Thread Management:** Manages threads and synchronization.
 - **Security Management:** Enforces security policies and permissions.
 - **Exception Handling:** Manages exceptions and ensures proper cleanup.

6. Application End:

- When the application finishes execution, the CLR performs cleanup tasks, such as finalizing objects and releasing resources.

The Default Thread

1. Entry Point Execution:

- The application's entry point, typically the Main method, is executed on the default thread, which is the main thread of the application.
- This main thread is created by the operating system when the application starts and is used by the CLR to run the initial code of the application.

2. Main Thread Characteristics:

- The main thread is responsible for executing the primary logic of the application.
- In console applications, the Main method runs on this default main thread.

- In GUI applications (like WinForms or WPF), the main thread also handles the UI event loop, processing user input and updating the user interface.

Types of Jit

Just-In-Time (JIT) compilation in the .NET environment (and in other runtime environments) can be categorized into different types based on when and how the compilation is performed. The three main types of JIT compilation are:

1. Pre-JIT (Pre-JIT Compilation)

- **Description:** Pre-JIT compiles all the code in the application into native code at the start of execution, before any method is executed.
- **When Used:** This type of JIT is typically used in scenarios where startup performance is critical, and all the code needs to be available in native form before execution starts.
- **Advantages:** Faster execution after the initial compilation since there is no runtime compilation delay.
- **Disadvantages:** Longer startup time due to the upfront compilation process.
- Pre-JIT compilation can be achieved using tools like **NGen** (Native Image Generator). NGen compiles IL code to native code ahead of time and stores the native images in a special cache on the disk. These native images are then loaded at runtime, bypassing the JIT compilation step.

2. Econo-JIT (Economy JIT)

- **Description:** Econo-JIT compiles methods into native code just before they are executed but does not retain the compiled code for future use. This means the method will be recompiled each time it is called.
- **When Used:** This type of JIT is used in resource-constrained environments where memory usage is a concern.
- **Advantages:** Lower memory usage since compiled code is not stored.
- **Disadvantages:** Slower execution for frequently called methods due to repeated compilation.

3. Normal-JIT (Standard JIT or JIT Compilation)

- **Description:** Normal-JIT compiles methods into native code just before they are executed and retains the compiled code in memory for future use. This is the most common type of JIT used in runtime environments.
- **When Used:** This type of JIT is used in general-purpose applications where a balance between startup time, memory usage, and execution speed is desired.
- **Advantages:** Faster execution for frequently called methods due to reuse of compiled code, balanced memory usage.

- **Disadvantages:** Initial execution of methods may be slower due to the compilation step.

Example of JIT Compilation in .NET

In .NET, the Normal-JIT (Standard JIT) is the default mode used by the CLR. Here's an illustration of how it works in a simple C# application:

```
using System;
```

```
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Hello, World!");
        int result = Add(5, 3);
        Console.WriteLine("Result: " + result);
    }
}
```

```
public static int Add(int a, int b)
{
    return a + b;
}
```

Execution Process:

1. **Startup:** The CLR loads the assembly and inspects the metadata.
2. **Main Method:** When the **Main** method is called, the JIT compiler compiles the IL code of **Main** into native code.
3. **Add Method:** When **Add** is called for the first time, the JIT compiler compiles the IL code of **Add** into native code.
4. **Execution:** The native code for **Main** and **Add** methods is executed.
5. **Reuse:** If **Add** is called again, the already compiled native code is executed, avoiding recompilation.

Summary

- **Pre-JIT:** Compiles all code before execution, resulting in faster runtime performance but longer startup times.
- **Econo-JIT:** Compiles code just before execution without retaining it, leading to lower memory usage but slower repeated execution.
- **Normal-JIT:** Compiles code just before execution and retains it, offering a balance between startup performance, runtime speed, and memory usage.

These JIT compilation types provide flexibility in optimizing different aspects of application performance, depending on the specific needs and constraints of the runtime environment.

(AOT) compilation

Ahead-of-Time (AOT) compilation in the context of .NET refers to the process of compiling Intermediate Language (IL) code to native machine code before the application is run, as opposed to Just-In-Time (JIT) compilation, which happens at runtime. AOT compilation effectively bypasses the JIT compilation process entirely, thus it doesn't fall into the typical JIT categories like Pre-JIT, Econo-JIT, or Normal-JIT. Instead, AOT is considered a distinct compilation strategy that aims to improve startup performance and reduce runtime overhead.

AOT Compilation Methods in .NET

1. NGen (Native Image Generator)

- **Description:** NGen compiles IL code into native code and stores the compiled images in the Native Image Cache on the target machine. This precompiled code is used at runtime instead of JIT-compiling the code on-the-fly.

```
ngen install MyApplication.exe
```

• **Characteristics:**

- Improves startup performance by reducing the amount of work the JIT compiler needs to do.
- Native images are specific to the machine they were generated on, which can affect portability.

2. ReadyToRun (R2R)

- **Description:** ReadyToRun is a form of AOT compilation that generates native code for your assemblies during the publish process. This native code is included alongside the IL code in the deployment package, allowing the runtime to use the precompiled native code when available.

```
dotnet publish -c Release -r win-x64 -p:PublishReadyToRun=true
```

- **Characteristics:**

- Enhances startup performance by using precompiled native code.
- Still maintains some flexibility since the IL code is also available, allowing for JIT compilation if necessary.

AOT vs. JIT Compilation

AOT Compilation:

- **Timing:** Compilation happens before runtime.
- **Performance:** Improved startup performance; potentially reduced runtime optimization.
- **Portability:** Native code is specific to the target platform; less portable than IL code.
- **Use Cases:** Suitable for scenarios where startup time is critical and the target environment is known and consistent.

JIT Compilation:

- **Timing:** Compilation happens at runtime, just before code execution.
- **Performance:** May have slower startup times due to runtime compilation; allows for runtime optimizations.
- **Portability:** IL code is platform-independent; highly portable across different environments.
- **Use Cases:** Suitable for general-purpose applications where portability and runtime optimizations are important.

Summary

AOT compilation, such as that provided by NGen and ReadyToRun, effectively bypasses the traditional JIT compilation process by precompiling IL code into native code before runtime. This improves startup performance and reduces runtime overhead. However, it should be noted that AOT compiled code might lack some of the dynamic optimizations that JIT compilation can provide based on runtime information.

AOT is distinct from traditional JIT types (Pre-JIT, Econo-JIT, Normal-JIT) and is used to achieve different performance characteristics, particularly in scenarios where startup time and predictable performance are critical.

Managed Code vs. Unmanaged Code

Managed code runs in the CLR environment and benefits from services like automatic memory management and security enforcement.

Unmanaged code runs outside the CLR environment and is typically written in languages like C or C++.

What is Exe and DLL? Assemblies and Namespaces

An Assembly is , one unit of .NET Application.

Note:- One Assembly can have any no of classes. and these classes can be logically grouped under namespaces.

So one Assembly can have zero or more namespaces.

one namespace is a logical collection of classes and sub namespaces

.NET Does not support primitive Data types

```
using System;

public class Class1
{
    static int Main(string[] args)
    {
        Console.WriteLine("Hello C#");
        return 0;
    }
}
```

```
myorg.cs

namespace MyOrg
{
    public class Class1
    {
        public string Hello()
        {
            return "Hello:Class1";
        }
    }
}

namespace Finance
{
    public class Class2
    {
        public string Greet()
        {
```

```
return "Greet:Class2";
}
}
namespace Payroll
{
public class Class3
{
public string Welcome()
{
return "Welcome:Class3";
}
}
}
}
}
```

```
csc -t:library myorg.cs
```

```
using MyOrg;
using MyOrg.Finance;
using System;
using Payroll= MyOrg.Finance.Payroll;
public class MainClass
{
static void Main()
{
Class1 c1 = new Class1();
string s1=c1.Hello();
Console.WriteLine(s1);

Console.WriteLine(new Class2().Greet());
Console.WriteLine(new Payroll.Class3().Welcome());

}
}
csc -r:Epam.dll Demo3.cs
```

```
using System;

public class Class1
{
static void Main()
{
```

```

Console.WriteLine("Enter the first number");
string s1=Console.ReadLine();
Console.WriteLine("Enter the second number");
string s2=Console.ReadLine();
int i1=Int32.Parse(s1);
int i2=int.Parse(s2);
Console.WriteLine((i1+i2));
}
}

```

C# Language basics

- C# Is Object Oriented
- C# is Case Sensitive.If follows the CamelCasing i.e.,Every C# name will start with capital letter and if that name include more than one word, then each word will start with the capital letter
- Comments
- C# is Strongly Typed Language
- Variable Declaration
`<datatype> <varname> = <default value>;`
- Method Declaration
`<returntype> <name>(<paramList>)`
`{`
`.....`
`}`

C# Supports 3 parameter Passing Mechanisms

In case of Pass by value (Default), the Parameter and Argument will have different memory locations. so the manipulations done on one will not affect the another.

In case of Pass by reference, the Parameter and Argument will have same memory location. so the manipulations done on one will affect the another.

PassByOut is very simillar to PassByref , but the difference is it can accept only output but not input

```

using System;

public class Class1
{
    public void Display(out int i)
    {
        i=20;
        i=i+1;
        Console.WriteLine(i);
    }
}

```

```
static void Main()
{
    int j=10;
    new Class1().Display(out j);
    Console.WriteLine(j);
}
}
```

Optional Parameters

```
public int Add(int a, int b = 0, int c = 0)
{
    return a + b + c;
}

// Call with all arguments
Console.WriteLine(calculator.Add(1, 2, 3)); // Output: 6

// Call with two arguments
Console.WriteLine(calculator.Add(1, 2)); // Output: 3 (c uses default value 0)

// Call with one argument
Console.WriteLine(calculator.Add(1)); // Output: 1 (b and c use default values 0)
}
```

Optional parameters must appear after all required parameters

```
public int Mul (int a = 0, int b , int c = 0)
{
    return a * (b * c);
}
```

Named Arguments

```
Console.WriteLine(calculator.Add(c:1, b:2, a:3));
```

1. When a method is written twice with the same parameter list , but differs in return type - is it overloading?
 1. Ex: public int Add(int i, int j)
 {
 return i + j;
 }
 public double Add(int i, int j)
 {
 return (double) i + j;
 }

No, by default it returns the error, saying it as a duplicate, it should differ in parameter signature in some way ex: double Add(int i, in int j) - Just like Out and Ref keyword, in keyword accepts only input, either you can use in or ref in this case if needed

2. In keyword is – pass by reference and read only
3. Out keyword – pass by reference and write only

Conditional structures

```
if(condition1)
{
...
}
else if(condition2)
{
....
}
...
else
{
....
}
```

```
switch(expr)
{
case v1:.....
break;
case v2:.....
break;
case v3:.....
break;
default:.....
}
```

Loop syntaxes

```
while(condition)
{
....
}

do
{
....
}while(condition)
```

```
for(initialization;condition;incr/decr)
{
.....
}

foreach(<object> o in <object collection>)
{
....
}
```

using Aliases

using a=System.Console

System namespace

```
Object (Supermost class)
  Console
  - ValueType
  - - SByte
  - Int16
Int32
Int64
Byte
UInt16
UInt32
UInt64
Single
Double
Array
Exception
String
Char
Boolean
Decimal
Datetime
....
```

Arrays

An array is a collection of similar datatype values

```
int[] i=new int[2];
i[0]=23;
i[1]=45;

(or)
```

```
int[] i=new int[]{23,45};
```

(or)

```
int[] i={23,45};
```

When we declare an array, it internally will be converted as one System.Array class Object.

Param Arrays

When we declare an array parameter as the paramarray, that parameter can accept any no of arguments of that parameter type.

```
using System;

public class Class1
{
    public int Add(params int[] ia)
    {
        int sum=0;
        for(int i=0; i<ia.Length;i++)
        {
            sum+=ia[i];
        }
        return sum;
    }
    static void Main(string[] args)
    {
        Class1 c1= new Class1();
        Console.WriteLine(c1.Add(12, 23, 45));
        Console.WriteLine(c1.Add(13,24,56,56,6,6,76,76));
    }
}
```

String formatting

```
using System;

public class Class1
{
    static void Main(string[] args)
    {
        try
        {
            string s1=args[0];
            int i1=Int32.Parse(s1);
```

```

string s2=args[1];
int i2=int.Parse(s2);
Console.WriteLine("The sum of " + i1 + " and " + i2 + " is:" + (i1+i2));
Console.WriteLine("The sum of {0} and {1} is: {2}", i1, i2, (i1+i2));

}
catch(IndexOutOfRangeException ex)
{
    Console.WriteLine(ex.Message);
}
catch(FormatException ex)
{
    Console.WriteLine(ex.Message);
}
catch(Exception ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    Console.WriteLine("in finally");
}
Console.WriteLine("response from API");
}
}

```

String verbatim

In C#, a verbatim string literal is a type of string literal that preserves **whitespace** and allows the **inclusion of escape characters** without the need for escaping them.

Verbatim string literals are prefixed with the **@** symbol.

```

using System;

class Program
{
    static void Main()
    {
        string multiLineString = @"
        This is a multi-line string.
        It preserves all white space
        and line breaks exactly as typed.";

        Console.WriteLine(multiLineString);
    }
}

```

```
}
```

String Interpolation

String interpolation in C# provides a more readable and convenient way to format strings compared to traditional methods like `String.Format`

Interpolated strings are prefixed with a `$` and allow embedding expressions inside curly braces `{}` directly within the string.

```
using System;

class Program
{
    static void Main()
    {
        string name = "John";
        int age = 30;

        string greeting = $"Hello, my name is {name} and I am {age} years old.";
        Console.WriteLine(greeting); // Output: Hello, my name is John and I am 30 years old.
    }
}
```

typeof Operator

```
using System;

public class Class1
{
    static void Main()
    {
        Type t = typeof(Class1);
        Console.WriteLine(t.Name);
        Console.WriteLine(t.Assembly.FullName);
    }
}
```

Ternary operator

The ternary operator in C# is a concise way to perform conditional evaluations. It is often used as a shorthand for simple if-else statements.

Nullable types in C#

In C#, nullable types are used to allow value types (such as `int`, `bool`, `DateTime`, etc.) to be assigned a value of `null`.

This is particularly useful when dealing with databases or other scenarios where a value might be undefined or missing.

To declare a nullable type, you append a `?` to the value type.

```
int? nullableInt = null;
bool? nullableBool = true;
DateTime? nullableDateTime = null;
```

You can use various properties and methods to work with nullable types

HasValue Property: Indicates whether the nullable type has a value

Value Property: Retrieves the value if it exists (throws an exception if it doesn't).

```
using System;

class Program
{
    static void Main()
    {
        int? nullableInt = null;

        if (nullableInt.HasValue)
            Console.WriteLine($"nullableInt has a value: {nullableInt.Value}");
        else
            Console.WriteLine("nullableInt is null.");

        int defaultInt = nullableInt != null ? nullableInt.Value : 0;
        Console.WriteLine(defaultInt); // Output: defaultInt: 0
    }
}
```

null-coalescing operator

The null-coalescing operator provides a default value if the left-hand operand is `null`.

```
int? nullableInt = null;
int defaultInt = nullableInt ?? 5;
```

Var vs dynamic

Feature	var	dynamic
---------	-----	---------

Type Determination	Compile-time	Runtime
Type Safety	Strong (compile-time checking)	Weak (runtime checking)
Initialization	Required at declaration	Not required at declaration
Type Change	Not allowed	Allowed
Performance	Faster (no runtime resolution)	Slower (runtime resolution needed)
Class-Level Fields	Not allowed	Allowed
Use Case	When type is known or can be inferred	When type can vary or is unknown at compile-time

References

- <https://dotnet.microsoft.com/en-us/>
- <https://visualstudio.microsoft.com/free-developer-offers/>
- <https://learn.microsoft.com/en-gb/dotnet/fundamentals/>
- https://learn.microsoft.com/en-gb/dotnet/core/introduction?WT.mc_id=dotnet-35129-website
- https://learn.microsoft.com/en-gb/training/dotnet/?WT.mc_id=dotnet-35129-website
- https://en.wikipedia.org/wiki/.NET_Framework
- https://en.wikipedia.org/wiki/.NET_Framework_version_history

- <https://en.wikipedia.org/wiki/.NET>

Assignment

- <https://learn.microsoft.com/en-gb/training/modules/csharp-write-first/>
- <https://learn.microsoft.com/en-gb/training/modules/dotnet-introduction/>
- <https://learn.microsoft.com/en-gb/training/modules/csharp-write-first/1-introduction>
- <https://learn.microsoft.com/en-gb/training/modules/csharp-literals-variables/>
- <https://learn.microsoft.com/en-gb/training/modules/csharp-basic-formatting/>
- <https://learn.microsoft.com/en-gb/training/modules/csharp-basic-operations/>
- <https://learn.microsoft.com/en-gb/training/modules/guided-project-calculate-print-student-grades/>
- <https://learn.microsoft.com/en-gb/training/modules/guided-project-calculate-final-gpa/>

- <https://learn.microsoft.com/en-us/training/paths/get-started-c-sharp-part-1/>
- <https://learn.microsoft.com/en-us/training/paths/get-started-c-sharp-part-2/>
- <https://learn.microsoft.com/en-us/training/paths/get-started-c-sharp-part-3/>
- <https://learn.microsoft.com/en-us/training/paths/get-started-c-sharp-part-4/>
- <https://learn.microsoft.com/en-us/training/paths/get-started-c-sharp-part-5/>
- <https://learn.microsoft.com/en-us/training/paths/get-started-c-sharp-part-6/>

- Given an array of binary digits, 0 and 1, sort the array so that all zeros are at one end and all ones are at the other. Which end does not matter. Determine the minimum number of swaps to sort the array.
Example: arr=[0,1,0,1,0]
With 1 move, switching elements 1 and 4, yields [0,0,0,1,1], a sorted array.
- Provide some Array members and sample examples

Properties: Length, Rank

Methods: Clear, Copy, CopyTo, IndexOf, LastIndexOf, Reverse, Sort, BinarySearch

```
using System;

public class ArrayExample
{
    public static void Main()
    {
        int[] numbers = { 5, 2, 9, 1, 5, 6 };

        // Length
        Console.WriteLine("Length: " + numbers.Length);

        // Rank
        Console.WriteLine("Rank: " + numbers.Rank);

        // IndexOf
        int index = Array.IndexOf(numbers, 9);
        Console.WriteLine("Index of 9: " + index);

        // Sort
        Array.Sort(numbers);
        Console.WriteLine("Sorted array: " + string.Join(", ", numbers));

        // Reverse
        Array.Reverse(numbers);
        Console.WriteLine("Reversed array: " + string.Join(", ", numbers));

        // Copy
        int[] copy = new int[numbers.Length];
        Array.Copy(numbers, copy, numbers.Length);
        Console.WriteLine("Copied array: " + string.Join(", ", copy));

        // Clear
        Array.Clear(copy, 0, copy.Length);
        Console.WriteLine("Cleared array: " + string.Join(", ", copy));
    }
}
```

- write a program of fibonacci series

```
using System;

public class FibonacciSeries
{
    public static void Main()
    {
```

```

    Console.WriteLine("Enter the number of terms for the Fibonacci series: ");
    int terms = int.Parse(Console.ReadLine());

    // Display the Fibonacci series
    PrintFibonacciSeries(terms);
}

public static void PrintFibonacciSeries(int terms)
{
    int first = 0, second = 1, next;

    Console.WriteLine("Fibonacci Series:");

    for (int i = 0; i < terms; i++)
    {
        if (i == 0)
        {
            Console.Write(first + " ");
            continue;
        }
        if (i == 1)
        {
            Console.Write(second + " ");
            continue;
        }
        next = first + second;
        first = second;
        second = next;
        Console.Write(next + " ");
    }
}
}

```

- Write a function that takes an integer i and returns a string with the integer backwards followed by the original integer.

123 → "321123" (Coding question)

```

using System;

public class ReverseAndAppend
{
    public static void Main()
    {
        int number = 123;
        string result = ReverseAndAppendInteger(number);
        Console.WriteLine(result); // Output: 321123
    }
}

```

```

public static string ReverseAndAppendInteger(int i)
{
    // Convert the integer to a string
    string original = i.ToString();

    // Reverse the string
    char[] charArray = original.ToCharArray();
    Array.Reverse(charArray);
    string reversed = new string(charArray);

    // Append the original string to the reversed string
    return reversed + original;
}
}

```

- Provide some string members and sample examples
 Properties: Length,
 Methods: IndexOf, LastIndexOf, Contains, Replace, ToUpper and ToLower, Trim, TrimStart, TrimEnd, Split, Join,

```

using System;

public class StringExample
{
    public static void Main()
    {
        string text = " Hello, World! ";

        // Length
        Console.WriteLine($"Length: {text.Length}");

        // Substring
        string sub = text.Substring(7, 5);
        Console.WriteLine($"Substring: {sub}");

        // IndexOf
        int index = text.IndexOf("World");
        Console.WriteLine($"Index of 'World': {index}");

        // Contains
        bool contains = text.Contains("Hello");
        Console.WriteLine($"Contains 'Hello': {contains}");

        // Replace
    }
}

```

```

string replaced = text.Replace("World", "C#");
Console.WriteLine($"Replaced: {replaced}");

// ToUpper and ToLower
string upper = text.ToUpper();
string lower = text.ToLower();
Console.WriteLine($"Uppercase: {upper}");
Console.WriteLine($"Lowercase: {lower}");

// Trim
string trimmed = text.Trim();
Console.WriteLine($"Trimmed: {trimmed}");

// Split
string[] words = text.Split(',');
Console.WriteLine("Split:");
foreach (string word in words)
{
    Console.WriteLine(word);
}

// Join
string joined = string.Join(", ", words);
Console.WriteLine($"Joined: {joined}");
}
}

```

- Reverse a string
- Find the second smallest element in unsorted array. You can't sort it.
- Given an interger array of size n filled with zero and ones ,
Arrange all zeros on left and all one on right side.
int a[n] = {01010101}
- Write a C# program to get all possible substrings from a string input
input: abcd
output: a ab abc abcd b bc bcd c cd d
- Provide Datetime members with examples
- Provide Object class members with examples
- Try different data conversion scenarios (ToString(), string to numeric types)
- Is, as operators

Self-Check

- What is Type Safe?
- what and how JIT compilation works
- What is CLR, CLS , CTS, CLI, BCL ?

- What is String Interpolation?
- What is string verbatim?
- Data Conversions
- What are value and reference types
- What is the difference between stack and Heap memory
- how to return multiple output from function
- What is the difference between string and string builder class
- What is boxing and unboxing
- var vs dynamic
- What are the main components of the .NET Framework?
- Explain the role of CLR in .NET Framework.
- What is JIT compilation? How does it work in .NET?
- What is the difference between value types and reference types in .NET?
- What is the purpose of namespaces in .NET?
- Explain the concept of assemblies in .NET.
- What are the different types of assemblies in .NET?
- What is the Global Assembly Cache (GAC)?
- What is the role of the Base Class Library (BCL) in .NET?
- What are the different types of application domains in .NET?
- What is the significance of the Main method in a C# application?
- Explain the difference between managed and unmanaged code.
- What is boxing and unboxing in .NET?
- What are the components of CLR?
- What is the difference between double and decimal?
- What are jagged arrays?